PSON5: A Portable, Layered Infrastructure for Explainable Personalization in AI Systems

Frederick Abila Independent Author — PSON5 Project github.com/fredabila/pson5

Version 1.0 · April 2026

Abstract

Large language model assistants are stateless by default. Every conversation begins with no memory of the user — their preferences, prior decisions, cognitive style, or the constraints under which earlier advice was given. Vendor-specific "memory" features partly address this, but they are opaque, non-portable, collapse observed evidence and model inference into a single store, and provide no mechanism for agent-facing reasoning over time. PSON5 is an open-source personalization infrastructure and file format that treats user state as a first-class, layered, confidence-annotated artifact rather than as free-form notes appended to a prompt. This paper describes the motivation, the three-layer data contract (observed, inferred, simulated) that is the core design invariant, the six-engine pipeline that implements it, the `.pson` file format, and the SDK surface through which agents consult profiles without reading raw internals. We report reference-workload latencies measured on the current TypeScript implementation — trait-merge at 5000-candidate scale, 1000-decision cold simulation, and full 1000-fact profile round-trip — all completing under 10 ms on commodity hardware, and we discuss the open problems that remain.

1. Introduction

An AI assistant that does not know the user it is speaking to produces one of two failure modes. The first is hedging: generic, widely applicable advice that could be copy-pasted to any user and carries no confidence in its fit. The second is false familiarity: confident personalization based on whatever the user happened to say in the first two turns, stripped of the context that would have made the personalization safe. Both are failures of infrastructure, not of model capability.

The current state of practice is to solve this in one of three ad-hoc ways: (i) vendor-specific memory features (e.g., ChatGPT Memory, Claude Projects), (ii) long-context prompts into which the caller manually copies previous conversations, or (iii) retrieval-augmented generation (RAG) pipelines over raw conversation logs. Each of these collapses a crucial distinction. A user statement *"I turned down a FAANG offer last year"* is evidence. A derived claim *"user values optionality over stability (confidence 0.71)"* is inference. A response to a

new question — *"user is likely to accept Series A founding engineer role"* — is a simulation. These three things have different epistemic status, different failure modes, and different safety requirements, but they are routinely mixed into a single chunked document or memory store.

PSON5 (pronounced "P-son-five") is a response to this. It specifies a portable file format, a six-engine runtime pipeline, and an SDK surface that together maintain the separation between what was observed, what was inferred, and what was predicted, while making the resulting profile consumable by any downstream model without model-specific adapters.

The contributions of this paper are: (1) a formal statement of the three-layer invariant as the hard constraint of the system; (2) a schema and file-format design (§4) for storing layered profiles with confidence metadata, consent scope, and reasoning provenance; (3) a reference implementation in TypeScript with measured latencies (§6); and (4) an explicit statement of the limitations and open design questions that remain (§7).

---

2. Background and Related Work

Opaque conversational memory. Proprietary memory systems in conversational assistants typically store free-form notes derived by the model itself. These notes have no structural distinction between direct user statements and model-generated summaries. A user inspecting their memory cannot, in general, tell which entries were said by them and which were inferred on their behalf. When the same memory is shared across products, as in multi-agent ecosystems, this ambiguity propagates downstream.

Vector memory and RAG. Embedding-based memory indexes raw text and retrieves it by semantic similarity. It preserves evidence verbatim but offers no mechanism for confidence decay, no explicit inference layer, and no simulation mechanism. It is a retrieval solution applied to a modeling problem.

Knowledge-graph approaches. Academic user-modeling literature has long advocated structured representations with explicit relations and confidence [1]. Industrial knowledge-graph infrastructure such as Neo4j provides the substrate but is agnostic to the epistemic distinctions PSON5 enforces.

User models in recommender systems. Classical recommender user models (e.g., collaborative filtering latent factors) already carry some of the ideas PSON5 generalises — probabilistic tendencies, decay, feedback loops — but they are optimised for ranking, not for consultation by an agent producing open-ended text.

PSON5 differs by treating the user model as a first-class artifact that (i) is portable across models and vendors, (ii) maintains layer separation as a non-negotiable invariant, (iii) carries confidence and evidence with every inferred claim, and (iv) exposes a simulation interface with explicit reasoning traces.

---

3. Design

3.1 The Three-Layer Invariant

The design invariant that organises the rest of the system is the separation of three layers within every profile:

1. **Observed** — direct user statements or direct behavioural evidence. Content in this layer was said or demonstrated by the user.
2. **Inferred** — model-derived traits, tendencies, and heuristics. Every entry must carry a confidence score and references to the observed evidence that supports it.
3. **Simulated** — scenario-specific predictions generated in response to a query. These are never written back into observed or inferred.

The scope document formalises this as the "Final Constraint" [2, §29]: *collapsing those three layers into one data model will damage trust, explainability, and safety*. All other design decisions — serialization format, engine boundaries, storage separation, API contracts — flow from preserving this invariant.

3.2 Pipeline

PSON5 defines six cooperating engines, plus auxiliary serialization and storage layers:

- **Acquisition Engine** collects signals via direct input, adaptive prompts, and optional behavioural capture. It also owns adaptive follow-up, fatigue detection, and contradiction probing.
- **Modeling Engine** transforms raw signals into structured attributes via three subsystems: a trait extractor (stable attributes), a pattern miner (repeated decisions), and a heuristic builder (conditional rules). Every derived claim is labelled with a confidence score and evidence references.
- **State Engine** tracks transient conditions — focused, distracted, stressed, motivated — each with trigger patterns, behaviour shifts, duration tendencies, and recovery signals.
- **Knowledge Graph Engine** maintains nodes and edges between learned entities and supports traversal queries. The reference deployment uses Neo4j; the adapter surface is sufficient for Postgres with pgvector or other property-graph backends.
- **Simulation Engine** answers four classes of query — decision simulation, response simulation, action likelihood, and counterfactual simulation — and produces explicit reasoning traces alongside every prediction.
- **Serialization Engine** writes and reads the portable `.pson` document.

3.3 The `.pson` File Format

A `.pson` file is a versioned JSON document whose top level contains a consent block, active-domain set, the three layers, dedicated cognitive / behavioural / state / knowledge-graph / simulation sub-models, privacy metadata, and system metadata (confidence, timestamps, revision). Required rules specify that the file must be versioned, observed and inferred fields must remain distinguishable, each inferred claim must support confidence

metadata, simulation outputs must be separable from durable profile facts, and privacy flags must travel with the profile.

3.4 Domain Modules

A `core + optional domains` model lets the base profile (identity, consent, cognitive, behavioural) be extended with first-party optional domains — productivity, education, finance, health, social — and developer-defined custom domains under a reserved namespace. Each domain supplies a schema fragment, a question set, supported depth levels, a sensitivity classification, and confidence rules.

3.5 Confidence, Evidence, and Decay

Every trait, pattern, and heuristic carries a confidence score, evidence references, a last-validated timestamp, and a decay policy. New evidence can strengthen or weaken existing claims. Older signals lose influence unless re-validated. Explicit user corrections carry high weight, and contradictions trigger targeted re-evaluation rather than silent override.

---

4. Implementation

The reference implementation is an npm-workspace monorepo in TypeScript, targeting Node ≥ 20 and MIT-licensed. Publishable packages under the `@pson5/` scope include `core-types`, `schemas`, `privacy`, `serialization-engine`, `provider-engine`, `modeling-engine`, `state-engine`, `graph-engine`, `neo4j-store`, `simulation-engine`, `acquisition-engine`, `agent-context`, `sdk`, `postgres-store`, and `cli`. Applications include a hand-rolled HTTP API, a terminal CLI (with an Ink-based console in progress), a documentation site, and a static landing page with a browser-based console.

Provider adapters. The `provider-engine` defines a `ProviderAdapter` interface and a registry that lets callers plug in any LLM provider. OpenAI, Anthropic, and OpenAI-compatible adapters ship as first-class; custom adapters require implementing three methods (`complete`, `extractTraits`, `proposeQuestions`). This isolates the rest of the system from vendor-specific details and keeps PSON5 model-agnostic in practice, not just in claim.

Zero-registry operation. A common deployment assumption is that a domain schema exists before profiling begins. PSON5 also supports a zero-registry mode: the acquisition engine asks an LLM to propose follow-up questions given the current state, and the modeling engine promotes AI-derived trait candidates into the inferred layer with `source = "ai_modeling"`. The downstream graph engine iterates all domains present in `layers.inferred`, so custom domains materialise without explicit registration.

Agent integration. The SDK exposes the profile to agents via a small surface — `load`, `observe`, `merge`, `simulate`, `projectContext` — plus an `agent-tools` module that packages PSON operations as tool definitions consumable by Claude, GPT, and OpenAI-compatible

models. The agent never reads the raw profile; it calls `projectContext(scope)` and receives a redacted, consent-scoped summary with a reasoning trace.

---

5. Evaluation

PSON5 ships a reproducible reference-workload script (`examples/remotion-teaser/scripts/benchmark.mjs`) that measures three workloads likely to lie on the hot path of any integration.

WORKLOAD	DESCRIPTION	MEDIAN	GOAL
Merge 5000 traits	Deduplicated merge of 2500 existing and 2500 incoming trait records	3.2 ms	50 ms
Simulate 1000 decisions	Weighted scoring of 1000 scenarios against 40 traits	1.2 ms	40 ms
Round-trip 1000 facts	Serialize + parse a 1000-fact layered profile	0.8 ms	20 ms

Samples are median-of-6 after two warmups, measured on a 2024 consumer laptop under Node 20. All three land more than an order of magnitude below their goal budgets. The numbers matter because agent-facing calls sit on the round-trip latency budget of the enclosing model call; anything above ~50 ms starts to be felt by users. PSON5's hot-path operations are therefore not the bottleneck.

The scope document also specifies a qualitative evaluation suite — prediction accuracy, confidence calibration, adaptability to behaviour change, profile completeness, and user burden (question fatigue) — along with a minimum test suite covering offline simulation, schema validation, API contract, privacy boundary, CLI workflow, and explainability integrity [2, §21]. A formal evaluation on real human subjects is out of scope for this paper and is part of future work.

---

6. Discussion

Why explainability belongs in the data model. If the only record of why an agent gave a piece of advice lives inside the model that produced it, the user has no mechanism for recourse. PSON5's inferred layer carries evidence references per claim; the simulation engine emits reasoning traces alongside predictions. A user or auditor can therefore answer the question *"why did the agent believe this?"* without asking the model to introspect.

Why layer separation is non-negotiable. A system that silently writes simulation outputs back into observed facts — i.e., treats its own guesses as evidence — will amplify its own biases on every subsequent turn. The three-layer invariant is the architectural guarantee that this cannot happen by accident. Every component in the pipeline is designed around it.

Trade-offs. Enforcing layer separation imposes schema and ergonomic costs. Callers must decide, at write time, which layer a piece of data belongs in. Provider adapters must tag AI-derived content with `source = "ai_modeling"` so downstream engines honour the boundary. The alternative — a single unstructured memory — is cheaper to build and use but, we argue, unsuitable for any deployment where the quality of personalization must be auditable.

---

7. Limitations and Future Work

Several limitations of the current implementation warrant direct acknowledgement.

First, the reference-workload benchmarks measure the engine's internal hot paths, not the end-to-end latency of a profile-grounded generation. A full round trip through the profile, SDK, provider adapter, and upstream model is dominated by the upstream model's time-to-first-token and is not materially influenced by PSON5's own overhead.

Second, no formal user study has yet evaluated prediction accuracy, confidence calibration, or perceived question burden against real human behaviour. These are precisely the metrics the scope document enumerates, and a targeted study is the next evaluation milestone.

Third, the privacy and consent model provides a framework (access levels, restricted fields, optional high-privacy mode) but not cryptographic enforcement of zero-knowledge deployment. A production-grade private-by-default deployment remains open work.

Fourth, domain-module authoring is currently a manual JSON-Schema exercise. An authoring tool that generates module fragments from example interactions would substantially lower the cost of extending the system.

Finally, the simulation engine's uncertainty quantification is presently calibrated heuristically. Replacing that with a Bayesian or conformal-prediction backbone would give simulations better-calibrated confidence, at the cost of additional runtime complexity.

---

8. Conclusion

PSON5 argues that personalization in AI systems is a data-modeling problem before it is a prompting problem. By enforcing a three-layer separation between what the user said, what the system inferred, and what the simulator predicted, and by carrying confidence, evidence, and consent with every piece of data, the resulting profile is portable across vendors, auditable by the user, and consumable by any model through a thin SDK. The reference implementation demonstrates that the required abstractions can be built without sacrificing hot-path latency. The harder, still-open problems — calibration, private-by-default deployment, and real-user evaluation — are questions of experimental rigour and engineering investment, not of architectural principle.

References

[1] G. I. Webb, M. J. Pazzani, and D. Billsus. **Machine Learning for User Modeling.** *User Modeling and User-Adapted Interaction*, 11(1–2):19–29, 2001.

[2] F. Abila. **PSON5 Scope — Build Reference (v5.0).** `PSON5_SCOPE.md`, April 2026. github.com/fredabila/pson5.

[3] Neo4j, Inc. **Cypher Query Language Reference.** neo4j.com/docs/cypher-manual.

[4] Anthropic. **Tool use with Claude.** Anthropic Developer Documentation, 2024–2026. docs.anthropic.com.

[5] OpenAI. **Memory and new controls for ChatGPT.** OpenAI Blog, February 2024. openai.com/index/memory-and-new-controls-for-chatgpt.

This whitepaper is distributed under the same MIT license as the PSON5 codebase. Corrections and critiques are welcomed via GitHub issues on the project repository.